

詳細設計書

LingoGlass AR — リアルタイム字幕表示 AR グラス

プロジェクト名	LingoGlass AR
ドキュメント ID	DD-LGA-001
バージョン	Rev 1.0 (S0実行承認版)
作成日	2026-05-12
対象フェーズ	詳細設計（内部設計） — Developer How
前提ドキュメント	要件定義書 REQ-LGA-001 Rev 1.0 / 基本設計書 BD-LGA-001 Rev 1.0
アーキテクチャ	レイヤードアーキテクチャ（Handler / Service / Repository） + モノレポ
次工程	WBS（WBS-LGA-001 予定） / テスト仕様書（TS-LGA-001 予定）

■ 改訂履歴

Rev	日付	担当者	変更内容
0.1	2026-05-12	NGUYEN MINH TRI	初版作成。BD-LGA-001 を入力としてモノレポ構成・クラス図・シーケンス図・インフラ設定を起草。

■ 目次

§ 1	本書の位置づけ・前提	p.2
§ 2	モノレポ構成（ディレクトリツリー）	p.2
§ 3	パッケージ責務・主要モジュール一覧	p.3
§ 4	ユースケース図 + UC 仕様書	p.3
§ 5	バックエンド クラス図（Node.js + WS）	p.4
§ 6	ファームウェア モジュール構成（ESP32-S3 / ESP-IDF）	p.5
§ 7	フロントエンド コンポーネント構成（Flutter）	p.6
§ 8	シーケンス図 — 会話字幕（正常系 / 異常系）	p.7
§ 9	シーケンス図 — OCR 翻訳 + IPO 図	p.8
§ 10	インフラ設定（fly.toml / Dockerfile / CI/CD）	p.8
	レビュー・承認欄	p.8

■ §1 本書の位置づけ・前提	
本書の対象	LingoGlass AR MVP の 内部実装方法 (HOW) を確定する。クラス構造・関数シグネチャ・エラーコード・環境変数・ディレクトリ配置までを定義し、開発者が手戻りなしに実装に入れる状態にする。
対象読者	Firmware / Mobile / Backend Engineer、QA、AI agents (コード生成補助)
記述レベル	関数シグネチャ・クラス図・状態遷移までを記述。実コード (メソッド本体) は対象外。
技術スタック (lock-in)	・ Firmware: ESP-IDF 5.x (C/C++17) on ESP32-S3 ・ Mobile: Flutter 3.22 / Dart 3.4 (iOS 16+ / Android 11+) ・ Backend: Node.js 20 LTS + Express 4 + ws 8 + TypeScript 5.4 ・ DB: PostgreSQL 16 (Drizzle ORM) / Redis 7 (任意) ・ Infra: Fly.io (東京リージョン nrt) / S3 互換ストレージ (Backblaze B2)
採用パターン	・ レイヤード: Handler → Service → Repository ・ 依存性注入: コンストラクタ DI (DI コンテナ未使用 — `index.ts` でワイヤー) ・ エラーは <code>AppError(code, message, status)</code> 統一型でレスポンスへ変換 ・ Flutter は features/ 単位の Riverpod ベース状態管理

■ §2 モノレポ構成（ディレクトリツリー）

```

lingoglass-ar/
├── apps/
│   ├── mobile/                                # Flutter app (iOS + Android)
│   │   ├── lib/
│   │   │   ├── main.dart
│   │   │   ├── app.dart                      # MaterialApp + Router
│   │   │   ├── features/
│   │   │   │   ├── pairing/                  # SCR-01
│   │   │   │   ├── home/                    # SCR-02
│   │   │   │   ├── conversation/            # SCR-03 (会話字幕)
│   │   │   │   ├── ocr/                     # SCR-04
│   │   │   │   ├── settings/                # SCR-05 + MOD-01/02
│   │   │   │   └── diagnostics/              # SCR-06
│   │   │   ├── core/
│   │   │   │   ├── ble/                     # BleClient, ConnectionManager
│   │   │   │   ├── api/                     # WsSessionClient, RestClient
│   │   │   │   ├── audio/                   # MicCapture, Vad, OpusEncoder
│   │   │   │   ├── camera/                  # CameraController, ImageCropper
│   │   │   │   └── storage/                  # SecureStore, AppPrefs
│   │   │   ├── shared/
│   │   │   │   ├── i18n/ ja.arb / en.arb / vi.arb
│   │   │   │   ├── theme/ tokens.dart
│   │   │   │   └── widgets/                  # 共通 UI (ChipStatus, etc.)
│   │   │   └── di/                          # Riverpod providers
│   │   ├── test/
│   │   └── pubspec.yaml
│   └── services/
│       ├── api/                                # Node.js + Express + ws
│       │   ├── src/
│       │   │   ├── handlers/                 # HTTP / WS エンドポイント
│       │   │   ├── services/                 # ビジネスロジック
│       │   │   ├── providers/                # 外部 AI ラッパー
│       │   │   ├── repositories/             # DB アクセス
│       │   │   ├── middleware/               # quota / device-auth / errors
│       │   │   ├── db/
│       │   │   │   ├── schema.ts             # Drizzle schema
│       │   │   │   └── migrations/
│       │   │   ├── utils/                   # logger, metrics
│       │   │   ├── index.ts                 # Composition root
│       │   │   └── env.ts                   # zod-validated env
│       │   ├── tests/
│       │   ├── Dockerfile
│       │   ├── fly.toml
│       │   └── package.json
│       └── firmware/
│           ├── esp32s3/                        # ESP-IDF C/C++
│           │   ├── main/
│           │   │   ├── main.c                # app_main
│           │   │   ├── ble_manager.[ch]      # BLE GATT サーバ
│           │   │   ├── packet_parser.[ch]    # $I0 プロトコル実装
│           │   │   ├── packet_assembler.[ch] # fragment 組立
│           │   │   ├── display_renderer.[ch]  # SPI/I2C 字幕描画
│           │   │   ├── battery_monitor.[ch]
│           │   │   ├── led_controller.[ch]    # ブライバシー LED
│           │   │   ├── power_manager.[ch]
│           │   │   └── ota_updater.[ch]       # OTA (NFR-12)
│           │   ├── components/
│           │   ├── CMakeLists.txt
│           │   └── sdkconfig.defaults
│           └── packages/
│               ├── shared-types/              # TypeScript 共通型 (ApiClient と Backend で共有)
│               │   ├── src/{api,ble,events}.ts
│               │   └── ble-protocol/          # BLE パケット仕様 (Dart + TS 両実装)
│               │       ├── ts/ PacketBuilder.ts / PacketParser.ts
│               │       └── dart/ packet_builder.dart / packet_parser.dart
│               └── infra/
│                   ├── docker-compose.yml     # 開発用 (api + postgres + redis)
│                   ├── postgres-init.sql
│                   ├── prometheus.yml
│                   ├── .github/workflows/
│                   │   ├── api-ci.yml         # build + test + deploy fly
│                   │   ├── mobile-ci.yml      # flutter test + lint
│                   │   └── firmware-ci.yml    # idf.py build
│                   └── docs/                  # 設計書 (本書含む)
│   ├── Makefile
│   ├── pnpm-workspace.yaml
│   └── README.md

```

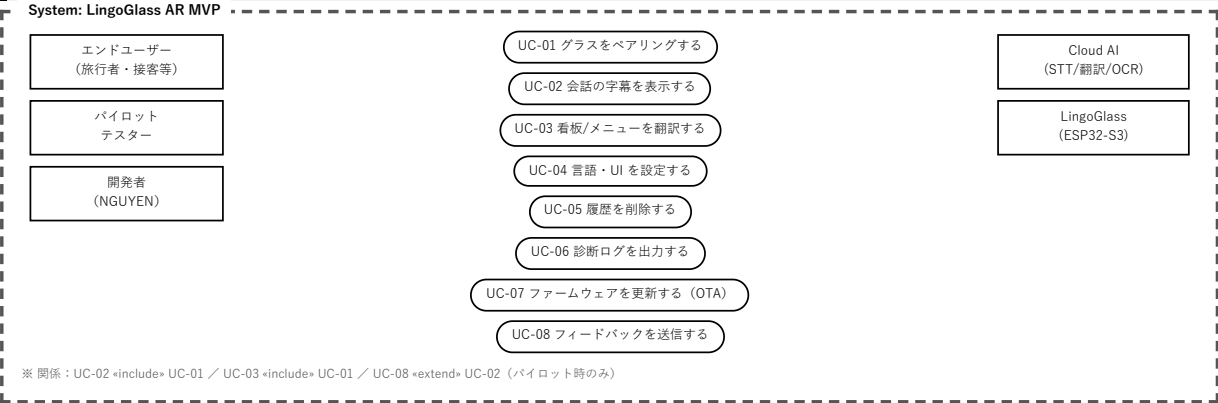
■ §3 パッケージ責務・主要モジュール一覧

パッケージ	言語	責務	主要依存	公開 I/F・備考
apps/mobile	Dart	6 画面 + BLE/API クライアント。ユーザー入力 → 音声/画像取得 → サーバー連携 → BLE 送信	flutter_blue_plus, just_audio, camera, web_socket_channel, drift	features/* の Riverpod Provider を Router 経由で公開
services/api	TS	WSS セッション / OCR / Logs / Feedback / Health の 7 API。Provider Router で STT・翻訳・OCR を抽象化	express, ws, drizzle-orm, zod, pino, openai, deepi-node, google-cloud/vision	docs/api-contract/openapi.yaml を /docs で配信。WSS は ws-events.schema.json と Zod で検証
firmware/esp32s3	C/C++	BLE GATT サーバ、字幕レンダリング、LED、バッテリー報告、OTA	esp-idf 5.x (nimble, lvgl 任意, spi_master)	カスタム GATT Service UUID (§10)
packages/ble-protocol	TS+Dart	BLE パケットの builder / parser を Mobile と Firmware で共有テスト	— (依存なし)	同一テストベクター (fixtures/*.json) で両実装を検証
packages/shared-types	TS	WSS メッセージ / REST DTO / event_type enum を Mobile (codegen) と API で共有	zod	codegen.ts で Dart 型を生成

3.1 Backend モジュール内訳 (services/api/src/)

レイヤー	モジュール	責務
handlers/	WsSessionHandler	WSS 接続受付・音声 chunk → SessionService に転送。session_id 発行。
	OcrHandler	POST /ocr。multipart → OcrService → JSON 返却。
	LogHandler / FeedbackHandler / HealthHandler	各 API のリクエスト変換と Service 呼び出し。
services/	SessionService	STT chunk → TranslateProvider → 結果ストリームをハンドラへ返す。タイムスタンプ記録。
	OcrService	OcrProvider → TranslateProvider。ocr_requests に履歴を記録 (画像 hash のみ)。
	EventService	session_events への一括書き込み・p50/p90/p95 集計。
providers/	SttProvider (interface)	実装: OpenAiRealtimeStt / FasterWhisperStt。STT_PROVIDER で切替。
	TranslateProvider (interface)	実装: OpenAiTranslate / DeepITranslate。
	OcrProvider (interface)	実装: GoogleVisionOcr。
repositories/	DeviceRepo / SessionRepo / EventRepo / OcrRepo / ErrorRepo / FeedbackRepo / TesterRepo	Drizzle ORM ラッパー。トランザクション境界はサービス層で管理。
middleware/	deviceAuth	X-Device-Id ヘッダを検証。MVP では存在チェックのみ。
	quotaMiddleware	device_id × 日次 1,000 リクエスト超過で 429 (FR-BE-04)。
	errorHandler	AppError → JSON エンベロープ、unknown → 500 + Sentry 通知。

■ §4 ユースケース図 + UC 仕様書

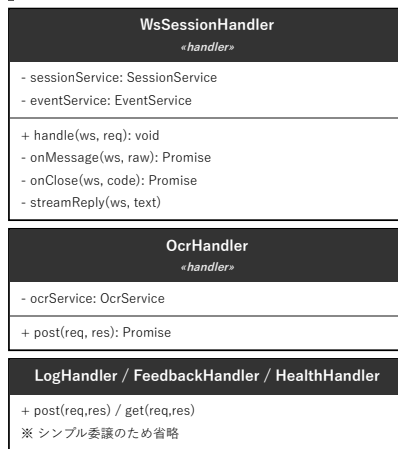


4.1 主要 UC 仕様書 (UC-02 会話字幕)

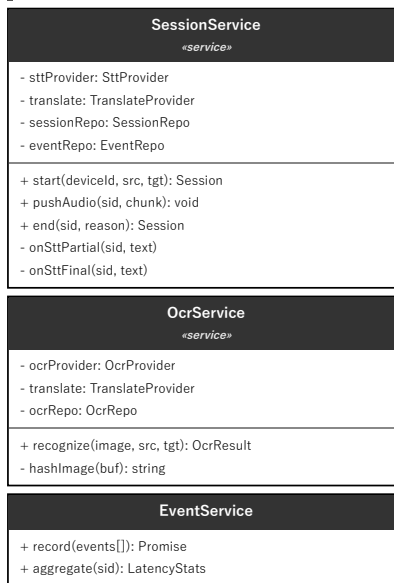
UC-ID	UC-02	名前	会話の字幕を表示する
主アクター	エンドユーザー	2 次アクター	Cloud AI (STT/翻訳)、LingoGlass
事前条件	グラスとペアリング済み (devices.is_active = true)、ネットワーク到達可能、マイク権限付与済み。		
基本フロー	① ユーザー SCR-03 を開く ② アプリが WSS /api/v1/session を確立し、sessions レコードを active で作成 ③ マイク收音 + VAD で発話区間を抽出し、Opus エンコードで chunk を送信 ④ サーバーが STT → Translate を逐次実行 ⑤ 翻訳テキストをアプリへ返却 ⑥ アプリが PacketBuilder で BLE フラグメントに分割 → グラスへ送信 ⑦ グラスが字幕レンダリング、ACK を返す ⑧ 各ステップの timestamp を session_events へ記録		
例外フロー	E1 WSS 切断 → 5 秒以内に再接続。失敗時 SCR-03 にエラーバーナ。 E2 STT/翻訳タイムアウト (5s) → エラー結果を BLE で「— 翻訳できませんでした —」として表示。 E3 BLE 切断 → 自動再接続 (最大 30s)。再接続中は字幕送信を抑止。		
事後条件	sessions.status='ended'、ended_at 設定。SCR-06 で p50/p90/p95 が確認できる。		

■ §5 バックエンド クラス図 (Node.js + WebSocket)

5.1 Handler 層



5.2 Service 層



5.3 Provider / Repository 層



5.4 主要型定義 (packages/shared-types)

```
// packages/shared-types/src/api.ts
export type ApiOk<T> = { success: true; data: T };
export type ApiFail = { success: false; error: { code: string; message: string } };
export type ApiResponse<T> = ApiOk<T> | ApiFail;

export type WsClientEvent =
  | { type: 'session.start'; deviceId: string; sourceLang: Lang; targetLang: Lang; retainText: boolean; clientTs: string }
  | { type: 'audio.chunk'; sessionId: string; seq: number; codec: 'opus16k'; dataBase64: string; clientTs: string }
  | { type: 'audio.end'; sessionId: string; seq: number; clientTs: string }
  | { type: 'metrics'; sessionId: string; name: string; valueMs: number; clientTs: string };
export type WsServerEvent =
  | { type: 'session.opened'; sessionId: string; serverTs: string }
  | { type: 'translation.partial'; sessionId: string; text: string; serverTs: string }
  | { type: 'translation.final'; sessionId: string; sourceText?: string; translatedText: string; durationMs: number; serverTs: string }
  | { type: 'error'; sessionId?: string; code: ErrorCode; message: string; retryable: boolean; serverTs: string };

export type OcrRequest = { image: File; sourceLang: Lang | 'auto'; targetLang: Lang; deviceId: string };
export type OcrResponse = { requestId: string; translatedText: string; ocrTextLength: number; durationMs: number };

export type Lang = 'ja' | 'en' | 'vi' | 'ko' | 'zh-CN';
export type ErrorCode =
  | 'STT_TIMEOUT' | 'STT_FAILED' | 'TRANSLATE_FAILED'
  | 'QUOTA_EXCEEDED' | 'INVALID_PAYLOAD' | 'INTERNAL';
```

5.5 エラーコード一覧

code	HTTP	発生元	意味・対処
STT_TIMEOUT	—	SttProvider 5s タイムアウト	WS event でクライアントへ通知。アプリは「翻訳できませんでした」を字幕送信
STT_FAILED	—	STT API エラー	同上。Sentry に severity=warn 記録
TRANSLATE_FAILED	—	翻訳プロバイダエラー	同上
QUOTA_EXCEEDED	429	quotaMiddleware	device_id × 日次 1,000 超過。Retry-After ヘッダ送出
INVALID_PAYLOAD	400	zod スキーマ検証	error.message に詳細パスを格納
DEVICE_NOT_FOUND	401	deviceAuth	X-Device-Id 未登録。再ペアリング誘導
PAYLOAD_TOO_LARGE	413	OcrHandler	画像 > 4 MB。クライアント側で再圧縮
INTERNAL	500	未捕捉例外	Sentry severity=error。ユーザーには汎用メッセージ

■ §6 ファームウェア モジュール構成 (ESP32-S3 / ESP-IDF 5.x)

BleManager «module» - gattService: nimble_t - connectedHandle: uint16_t - mtu: int (20/185/247) + ble_init(void): esp_err_t + ble_start_adv(void): void + ble_send_notify(buf, len): esp_err_t - on_connect(handle, addr) - on_write(handle, buf, len) - on_disconnect(reason)	PacketAssembler - current_seq: uint16_t - received_frags: bitmap - buffer: char[300] - deadline_ms: int + feed(pkt): assemble_state_t + is_complete(): bool + get_text(out): size_t + tick(now_ms) // 5s timeout + discard_old(new_seq)	LedController + led_init() + led_on() // 収音/撮影中 + led_off() + led_blink(period_ms) BatteryMonitor + read_voltage_mv(): int + to_percent(mv): int + publish_notify() // 10% step PowerManager + enter_idle_dim() // 30s 無活動 + enter_sleep() // 60s 無活動 + wake_on_ble() OtaUpdater + ota_begin(total): esp_err_t + ota_write(chunk, n) + ota_end_and_reboot()
PacketParser + parse(buf, len, out): parse_result_t + crc8(buf, len): uint8_t	DisplayRenderer - spi_dev: spi_device_handle_t - font_atlas: const Font* + display_init(): esp_err_t + display_render(text): void + display_clear(): void + display_set_dim(level): void	

6.1 状態遷移 (PowerManager / 字幕表示)

[OFF] --power on--> [BOOT] --ble_init ok--> [ADVERTISING] [ADVERTISING] --connect--> [READY] [READY] --subtitle pkt--> [DISPLAYING] (字幕表示) [DISPLAYING] --subtitle clear / 30s--> [READY] [READY] --60s 無活動--> [SLEEP] (display off, BLE notify keep) [SLEEP] --pkt received--> [READY] (wake) [ANY] --disconnect--> [ADVERTISING] (自動再接続待ち) [ANY] --battery<5%--> [LOW_POWER] (字幕は維持、輝度 50%)

6.2 IPO 図 ― 字幕パケット受信処理 (main loop)

Input	Process	Output
BLE write characteristic からのバイナリパッ ファ (最大 MTU - 3 bytes)	① CRC8 検証 (失敗 → NAK 0x31 返信) ② version, type, seq, frag_index, frag_count, payload_len を解釈 ③ PacketAssembler.feed(pkt) ④ 新 sequence_id → 旧進行中破棄 (discard_old) ⑤ 全 fragment 揃ったら DisplayRenderer.render(text) ⑥ ACK 0x30 を返信	① 字幕がディスプレイへ描画される ② ACK / NAK を BLE notify で送信 ③ 5 秒タイムアウト: sequence 破棄ログ (任意)
5 秒経過した PacketAssembler のタイマー tick	① is_complete() false かつ deadline 超過 → assembler reset ② Optionally Send STATUS_REPORT (timeout) to App	① 壊れた fragment セットが破棄される ② アプリ側は再送せず、新しい字幕の到着を待つ

■ §7 フロントエンド コンポーネント構成 (Flutter)

7.1 Widget ツリー (features/ 単位)

```
App (MaterialApp.router)
├── GoRouter
│   ├── /onboarding → PairingScreen (features/pairing)
│   │   └── BleDeviceList → DevicePairButton
│   ├── /home → HomeScreen (features/home)
│   │   ├── ConnectionStatusBar
│   │   └── ModeGrid (Conversation / Ocr / Settings / Diagnostics)
│   ├── /translate/voice → ConversationScreen (features/conversation)
│   │   ├── ListeningIndicator (LED 連動)
│   │   ├── SubtitleLabel (current + history)
│   │   ├── PttButton
│   │   └── HistoryToggle (default OFF)
│   ├── /translate/ocr → OcrScreen (features/ocr)
│   │   ├── CameraPreview
│   │   ├── CaptureButton
│   │   └── ResultPreview → SendToGlassButton
│   ├── /settings → SettingsScreen (features/settings)
│   │   ├── LangSelector ×2 (src / tgt) → showLangModal()
│   │   ├── UiLangSelector (ja/en/vi)
│   │   ├── PttToggle
│   │   ├── HistoryToggle (default OFF)
│   │   └── PrivacyActions (history view / purge / unpair)
│   └── /diagnostics → DiagnosticsScreen (features/diagnostics)
│       ├── LatencyChart (p50/p90/p95)
│       ├── GlassStatusCard (battery / temp / ble)
│       ├── ErrorLogList
│       └── ExportActions (CSV / JSON / upload)
```

7.2 状態管理 (Riverpod プロバイダ)

Provider	型	参照する Core モジュール	役割
bleConnectionProvider	StateNotifier<BleState>	core/ble/BleClient	connected / disconnected / pairing 状態を全画面で購読
activeDeviceProvider	FutureProvider<Device?>	core/storage/SecureStore	SecureStorage の device_id を提供
conversationSessionProvider	AsyncNotifier<Session>	core/api/WsSessionClient	SCR-03 のセッション状態・受信テキスト
ocrResultProvider	AsyncNotifier<OcrResult?>	core/api/RestClient	SCR-04 撮影後の結果保持
settingsProvider	Notifier<Settings>	core/storage/AppPrefs	言語 / PTT / 履歴 / UI 言語
latencyStatsProvider	FutureProvider<Stats>	core/api/RestClient	SCR-06 用 p50/p90/p95
i18nProvider	Provider<Localizations>	shared/i18n	UI 言語切替の即時反映

7.3 Core クライアントクラス

BleClient «core» - _scanner: FlutterBluePlus - _connection: BluetoothDevice? - _subtitleChar: Characteristic? - _packetBuilder: PacketBuilder + scan(): Stream<ScanResult> + connect(deviceId): Future + sendSubtitle(text): Future + disconnect(): Future + state(): Stream<BleState>	MicCapture + Vad + start(): Stream<Int16List> + stop() - detectVoice(frame): bool - emitSegment(segment)
WsSessionClient - _channel: WebSocketChannel - _eventCtrl: StreamCtrl<WsEvent> + open(deviceId, src, tgt) + pushAudio(bytes): void + events(): Stream<WsEvent> + close(reason): Future	RestClient + postOcr(image, src, tgt): OcrResult + getEvents(sid): List<Event> + purge(deviceId): void + postFeedback(input): void + postLogs(events[]): void
	PacketBuilder (packages/ble-protocol) + build(text, seqId, mtu): List<Packet> - crc8(buf): int + chunkSize(mtu): int

7.4 画面の主要状態遷移 (SCR-03 ConversationScreen)

```
[IDLE] --マイク許可 OK--> [READY]
[READY] --WS 接続 OK--> [LISTENING] (LED ON、收音中)
[LISTENING] --VAD 発話--> [TRANSCRIBING] (chunk 送信中)
[TRANSCRIBING] --stt.final 受信--> [TRANSLATING]
[TRANSLATING] --translate.final--> [SENDING_BLE]
[SENDING_BLE] --ACK 受信 or 失敗--> [LISTENING] (次発話を待つ)
[ANY] --WS 切断--> [RECONNECTING] (5s リトライ)
[ANY] --BLE 切断--> [BLE_RECONNECTING] (LED 点滅、字幕送信停止)
[ANY] --ユーザー停止 / 戻る--> [IDLE]
```

■ § 8 シーケンス図 — 会話字幕表示

8.1 正常系（Happy Path） — UC-02

User	Flutter App (SCR-03)	WsSession Handler	Session Service	SttProvider + Translate	PostgreSQL (sessions/events)	LingoGlass (ESP32-S3)
SCR-03 を開く	→WSS open	受付				
		→start()	create()		INSERT sessions status='active'	
	←-- session.opened {sid}					
発話	VAD 切出し → audio chunk	受信	pushAudio()	→stt.open()		
	←-- stt.partial	stream		←-- chunk		
	←-- stt.final		onSttFinal()	→translate()	INSERT event 'stt_done'	
	←-- translate.final			←-- text	INSERT event 'translate_done'	
	PacketBuilder.build() → BLE notify					fragment 受信 → assemble
字幕を読む	←-- ACK				INSERT 'ble_sent' + 'displayed'	DisplayRenderer .render()
	loop [会話が続く間] — 4〜9 を繰り返す					
画面を閉じる	→close()	end(sid)	updateStatus()		UPDATE status ='ended'	字幕消去 (SUBTITLE_CLEAR)

8.2 異常系（Error Path） — STT タイムアウト + BLE 切断

User	Flutter App	WsHandler	SessionService	SttProvider	PostgreSQL / Sentry	LingoGlass
発話	→audio chunk	受信	pushAudio	stt.open()		
				… 5s 無応答 …		
	alt [STT_TIMEOUT]					
	←-- error STT_TIMEOUT	stream	catch + log		INSERT error_log severity='warn'	
	BLE 送信 "— 翻訳できませんでした —"					エラー字幕 表示
	alt [BLE 切断（送信中）]					
	BleClient.state → disconnected SCR-03 にバナー表示					切断
	自動再接続 （最大 30s, 指数バックオフ）					advertising
	再接続成功				INSERT event 'ble_reconnected'	connected

※ STT_TIMEOUT は最大 3 連続で発生した場合のみ Sentry severity を error に昇格。BLE 切断 30 秒以上で UC-02 を中止し、ユーザーに「ペアリングを確認してください」を表示。

■ §9 シーケンス図 — OCR 翻訳 (UC-03 正常系)

User	Flutter App (SCR-04)	OcrHandler	OcrService	GoogleVisionOcr + Translate	PostgreSQL / LingoGlass
撮影	撮影 → クロップ → POST /api/v1/ocr (multipart)	受信 + zod 検証	recognize()		
			hashImage(SHA-256) + INSERT ocr_requests		INSERT ocr_requests (image_hash のみ)
			→visionOcr()	OCR テキスト抽出	
			→translate()	翻訳結果	
レビュー確認	←- 200 OcrResult	JSON 返却	UPDATE status='success'		UPDATE duration_ms
「グラスへ送信」	PacketBuilder → BLE notify				字幕表示

■ §10 インフラ設定 (fly.toml / Dockerfile / CI/CD)

10.1 services/api/fly.toml

```
app = "lingoglass-api"
primary_region = "nrt" # 東京

[build]
  dockerfile = "Dockerfile"

[env]
  NODE_ENV = "production"
  PORT = "8080"
  STT_PROVIDER = "openai-realtime"
  TRANSLATE_PROVIDER = "deepl"
  OCR_PROVIDER = "google-vision"
  LOG_LEVEL = "info"

[[services]]
  internal_port = 8080
  protocol = "tcp"
  [[services.ports]]
    handlers = ["tls", "http"]
    port = 443

[[services.tcp_checks]]
  interval = "30s"
  timeout = "5s"
  grace_period = "10s"

[[vm]]
  cpu_kind = "shared"
  cpus = 1
  memory_mb = 512

# Secrets (fly secrets set ...)
# OPENAI_API_KEY=...
# DEEPL_API_KEY=...
# GOOGLE_APPLICATION_CREDENTIALS_JSON=...
# DATABASE_URL=postgres://...
# REDIS_URL=redis://...
# SENTRY_DSN=...
```

10.2 services/api/Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY pnpm-lock.yaml package.json ./
RUN corepack enable && pnpm install --frozen-lockfile
COPY . .
RUN pnpm build

FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY package.json ./
ENV NODE_ENV=production
EXPOSE 8080
USER node
HEALTHCHECK --interval=30s --timeout=5s \
  CMD wget -qO- http://localhost:8080/api/v1/healthz || exit 1
CMD ["node", "dist/index.js"]
```

10.3 GitHub Actions (CI/CD)

Workflow	トリガー	主要ステップ
.github/workflows/api-ci.yml	push / PR (paths: services/api/**)	① pnpm install ② lint (eslint) ③ typecheck ④ test (vitest) ⑤ build ⑥ docker build (PR は build のみ) ⑦ main へ push 時 flyctl deploy --remote-only
.github/workflows/mobile-ci.yml	push / PR (paths: apps/mobile/**)	① flutter pub get ② flutter analyze ③ flutter test ④ build apk (PR で attach as artifact) ⑤ TestFlight / Internal Testing は手動トリガー
.github/workflows/firmware-ci.yml	push / PR (paths: firmware/**)	① idf.py build (espressif/idf:release-v5.2 コンテナ) ② size check (フラッシュ閾値 80%) ③ artifact として bin をアップロード ④ OTA リリースは v* tag で手動
.github/workflows/contract-test.yml	push (paths: packages/ble-protocol/**)	TS 実装と Dart 実装で同一 fixtures/*.json をパース → 結果の同一性を検証

10.4 監視・運用

用途	ツール	設定値
APM / エラー監視	Sentry	severity: info/warn/error/critical。release は git sha
メトリクス	Prometheus + Grafana (任意)	API レイテンシ p50/p95、WS 接続数、quota 使用率を計測
コスト監視	Cost Monitor (自前 cron)	毎日 OpenAI / DeepL / Vision の API usage を集計、\$550/月 cap に達したら Slack 通知
バックアップ	cron + flyctl postgres backup	毎日 03:00 JST に dump → B2 へ転送 (F-06)

■ レビュー・承認欄

作成者

氏名：NGUYEN MINH TRI

日付：2026-05-12

署名：_____

レビュー者

氏名：_____

日付：_____

署名：_____

承認者

氏名：_____

日付：_____

署名：_____

※ Rev 1.0 としてS0実行を承認済み。次工程「WBS」(DOC-ID: WBS-LGA-001 予定) および「テスト仕様書」(TS-LGA-001 予定) へ移行する。